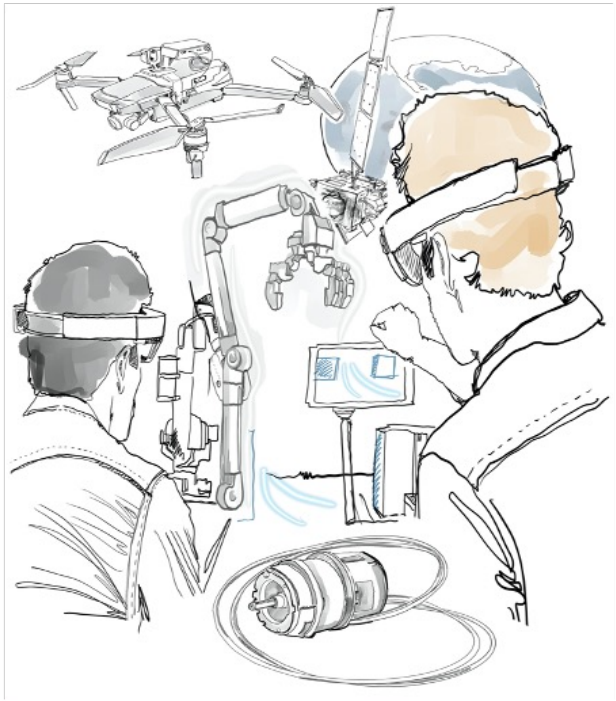
Robotic Perception and Action

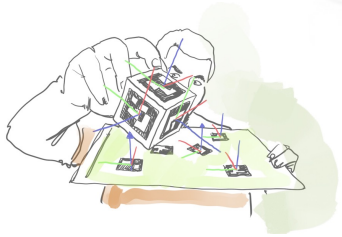
Sensor fusion with complementary filters

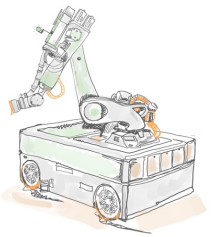




Mariolino De Cecco

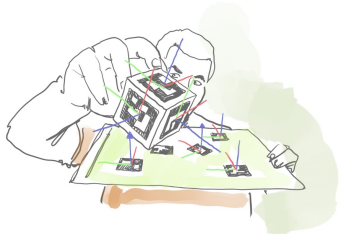
Laboratory website: http://www.miro.ing.unitn.it/

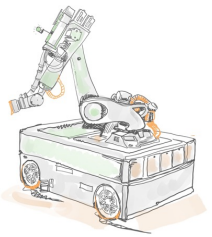




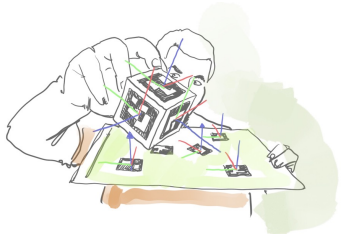
Sources

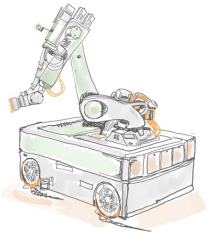
- Doebelin
- Exercises in class on the baro-inertial attitude estimation
- Iop 2003





CONVENTIONS





$$\mathbf{X}_k = \begin{bmatrix} x_k \\ y_k \\ \delta_k \end{bmatrix}$$

$$T_c = t_k - t_{k-1}$$

R_R, R_L: right and left wheel radii;

$\omega_R(t), \omega_L(t)$: right and left wheel angular velocities;

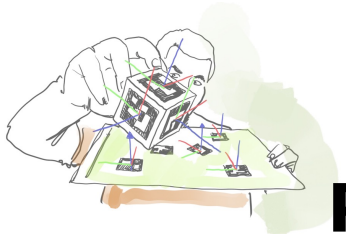
b: distance between the two driving wheels;

$v(t)$: longitudinal velocity of the robot reference point P_r;

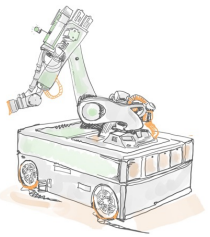
$\dot{\delta}(t)$: robot yaw rate;

$\delta(t)$: robot attitude;

$x(t), y(t)$: position of P_r in the fixed reference frame.

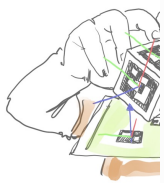


Pose and conventions



1. General notation and robot pose

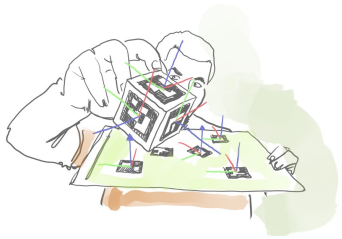
Symbol	Meaning
k	Current discrete-time sample / Perception loop
$k-1$	Previous discrete-time sample
t_k	Time corresponding to sample k
$T_c = t_k - t_{k-1}$	Perception-loop sampling period
x_k	Robot x-position at sample k
y_k	Robot y-position at sample k
δ_k	Robot attitude / yaw angle at sample k
$\mathbf{X}_k = [x_k \; y_k \; \delta_k]^T$	Robot pose vector
P_r	Reference point of the robot
b	Distance between right and left driving wheels
$\Delta(\cdot)_k$	Increment of a quantity during $t_{k-1} \rightarrow t_k$

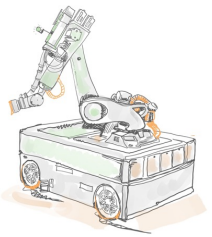




2. Superscripts

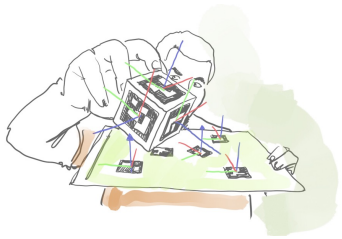
Superscript	Meaning
E	Encoder / odometric estimate
G	Gyroscope estimate
H	HTC absolute measurement
EG	Encoder–gyroscope fused estimate
F	Final fused estimate
HP	High-pass filter / branch
LP	Low-pass filter / branch

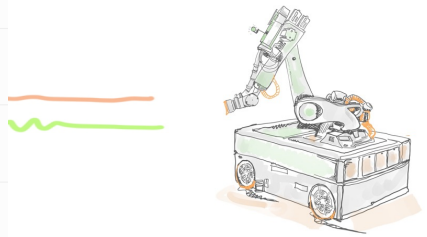




3. Wheel encoders and differential-drive kinematics

Symbol	Meaning
R_R	Right-wheel radius
R_L	Left-wheel radius
N_R	Right encoder counts per wheel revolution
N_L	Left encoder counts per wheel revolution
$n_{R,k}$	Right encoder cumulative count at sample k
$n_{L,k}$	Left encoder cumulative count at sample k
$\Delta n_{R,k}$	Right encoder count increment
$\Delta n_{L,k}$	Left encoder count increment
$\omega_R(t)$	Right-wheel angular velocity
$\omega_L(t)$	Left-wheel angular velocity
$v_R(t) = R_R\omega_R(t)$	Right-wheel linear velocity
$v_L(t) = R_L\omega_L(t)$	Left-wheel linear velocity
$v(t)$	Robot longitudinal velocity
$\dot{\delta}(t)$	Robot yaw rate
$\Delta s_{R,k}$	Signed right-wheel displacement
$\Delta s_{L,k}$	Signed left-wheel displacement
Δs_k^E	Encoder estimate of robot longitudinal displacement
$\Delta \delta_k^E$	Encoder estimate of attitude increment

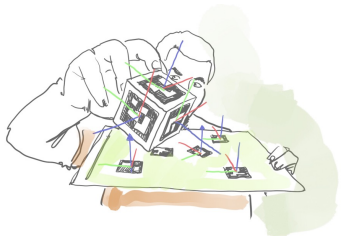


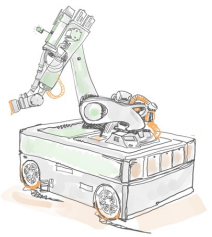


4. Gyroscope

Symbol	Meaning
$\omega_{G,k}$	Gyroscope measured yaw rate
b_G	Gyroscope zero-rate bias
$\Delta\delta_k^G$	Gyroscope estimate of attitude increment

$$\Delta\delta_k^G = T_c(\omega_{G,k} - b_G)$$

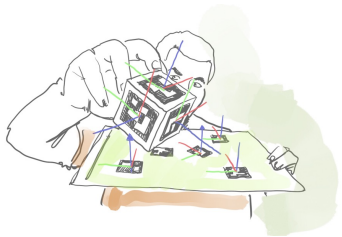




5. Maneuver-dependent encoder–gyro fusion

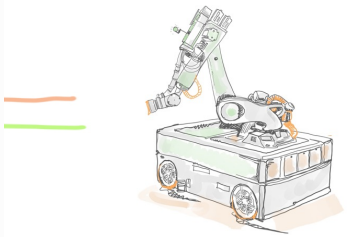
Symbol	Meaning
μ_k	Continuous turning / differential-wheel-motion index
ρ_k	Continuous vehicle-motion index
ε	Small positive constant preventing division by zero
$\sigma^E_{\Delta\delta,k}$	Standard uncertainty of encoder attitude increment
$\sigma^G_{\Delta\delta,k}$	Standard uncertainty of gyro attitude increment
$\xi_{R,k}$	Encoder-to-gyro attitude accuracy/uncertainty ratio
$f_\rho(\cdot)$	Continuous function defining ρ_k
$f_\xi(\cdot)$	Continuous function defining $\xi_{R,k}$
$w_{E,k}$	Encoder weight in attitude-increment fusion
$w_{G,k}$	Gyroscope weight in attitude-increment fusion
$\Delta\delta^{EG}_k$	Encoder–gyro fused attitude increment

$$\xi_{R,k} = \frac{\sigma^E_{\Delta\delta,k}}{\sigma^G_{\Delta\delta,k}}$$



$$w_{E,k} = \frac{1}{1+\xi^2_{R,k}}, \qquad w_{G,k} = \frac{\xi^2_{R,k}}{1+\xi^2_{R,k}}$$

$$\Delta\delta^{EG}_k = w_{E,k}\Delta\delta^E_k + w_{G,k}\Delta\delta^G_k.$$



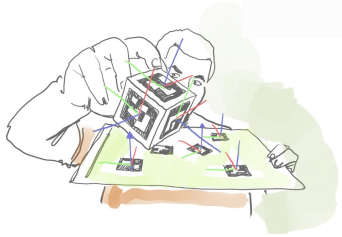
6. Attitude estimates

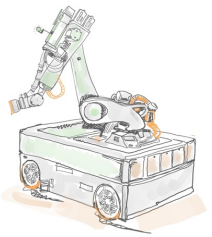
Symbol	Meaning
δ_k^{EG}	Relative attitude obtained by integrating $\Delta\delta^{EG}$
δ_k^{H}	Absolute attitude measured by HTC
δ_k^{F}	Complementary-filter fused attitude
$\Delta\delta_k^{F}$	Increment of fused attitude
$\bar{\delta}_k^{F}$	Representative / mean fused attitude over $k-1 \to k$

Useful definitions:

$$\Delta\delta_k^F = \mathbf{wrap}\left(\delta_k^F - \delta_{k-1}^F\right)$$

$$\bar{\delta}_k^F = \delta_{k-1}^F + \frac{\Delta\delta_k^F}{2}.$$





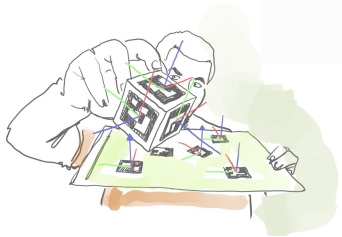
7. Position estimates

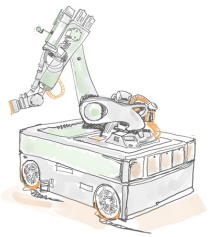
Symbol	Meaning
x_k^E, y_k^E	Encoder dead-reckoning position estimate
x_k^H, y_k^H	Absolute HTC position measurement
x_k^F, y_k^F	Complementary-filter fused position
$\mathbf{p}_k^E = [x_k^E \; y_k^E]^T$	Encoder position vector
$\mathbf{p}_k^H = [x_k^H \; y_k^H]^T$	HTC position vector
$\mathbf{p}_k^F = [x_k^F \; y_k^F]^T$	Fused position vector

Encoder position propagation:

$$x_k^E = x_{k-1}^E + \Delta s_k^E \cos \bar{\delta}_k^F$$

$$y_k^E = y_{k-1}^E + \Delta s_k^E \sin \bar{\delta}_k^F .$$





8. Continuous complementary-filter notation

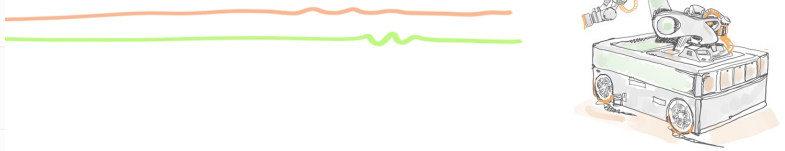
Symbol	Meaning
s	Laplace-transform variable
$H_{\delta}^{HP}(s)$	Attitude high-pass filter
$H_{\delta}^{LP}(s)$	Attitude low-pass filter
$H_{p}^{HP}(s)$	Position high-pass filter
$H_{p}^{LP}(s)$	Position low-pass filter
$A_{\delta}(s)$	Common denominator polynomial for attitude filters
$B_{\delta}^{HP}(s)$	HP numerator polynomial for attitude
$B_{\delta}^{LP}(s)$	LP numerator polynomial for attitude
$A_{p}(s)$	Common denominator polynomial for position filters
$B_{p}^{HP}(s)$	HP numerator polynomial for position
$B_{p}^{LP}(s)$	LP numerator polynomial for position
$a_{\delta,1}, a_{\delta,2}, a_{\delta,3}$	Continuous attitude-filter denominator coefficients
$a_{p,1}, a_{p,2}, a_{p,3}$	Continuous position-filter denominator coefficients
$f_{c,\delta}$	Attitude complementary-filter cutoff frequency [Hz]
$f_{c,p}$	Position complementary-filter cutoff frequency [Hz]
$\omega_{c,\delta} = 2\pi f_{c,\delta}$	Attitude cutoff angular frequency
$\omega_{c,p} = 2\pi f_{c,p}$	Position cutoff angular frequency

Complementarity:

$$H^{HP}(s) + H^{LP}(s) = 1$$

and, for the selected third-order construction,

$$B^{HP}(s) = s^3, \qquad B^{LP}(s) = A(s) - B^{HP}(s).$$



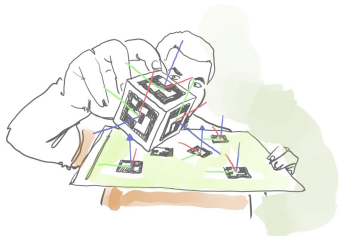
9. Discrete complementary-filter notation

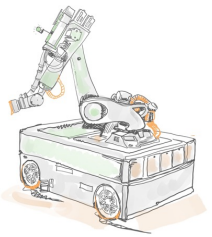
Symbol	Meaning
z	Z-transform variable
z^{-1}	One-sample delay
$H^{HP}(z)$	Discretized high-pass transfer function
$H^{LP}(z)$	Discretized low-pass transfer function
a_i^d	Discrete denominator coefficient
b_i^{HP}	Discrete HP numerator coefficient
b_i^{LP}	Discrete LP numerator coefficient
$i = 0, \ldots, 3$	Numerator coefficient index
$i = 1, \ldots, 3$	Denominator coefficient index

Generic third-order form:

$$H^{HP}(z) = \frac{b_0^{HP} + b_1^{HP} z^{-1} + b_2^{HP} z^{-2} + b_3^{HP} z^{-3}}{1 + a_1^d z^{-1} + a_2^d z^{-2} + a_3^d z^{-3}}$$

$$H^{LP}(z) = \frac{b_0^{LP} + b_1^{LP} z^{-1} + b_2^{LP} z^{-2} + b_3^{LP} z^{-3}}{1 + a_1^d z^{-1} + a_2^d z^{-2} + a_3^d z^{-3}}.$$





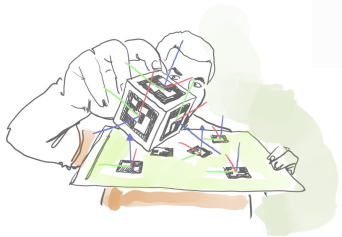
10. Internal filter states / outputs

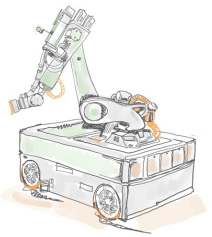
These are useful to include because they appear explicitly in the C++ recurrence:

Symbol	Meaning
δ_k^{HP}	HP-filtered encoder–gyro attitude
δ_k^{LP}	LP-filtered HTC attitude
x_k^{HP}	HP-filtered encoder x estimate
x_k^{LP}	LP-filtered HTC x estimate
y_k^{HP}	HP-filtered encoder y estimate
y_k^{LP}	LP-filtered HTC y estimate

With

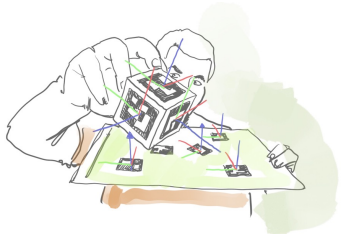
$$\delta_k^F = \delta_k^{HP} + \delta_k^{LP},$$
$$x_k^F = x_k^{HP} + x_k^{LP},$$
$$y_k^F = y_k^{HP} + y_k^{LP}.$$

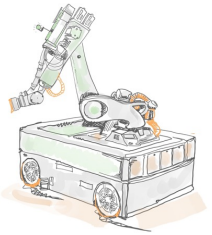




kinematics

THE ROBOTIC WALKER





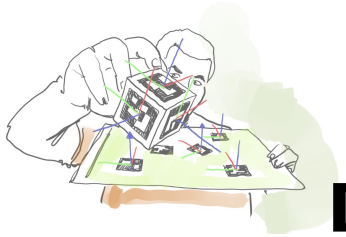
The linear and angular motion kinematics are separate in a differential drive vehicle:

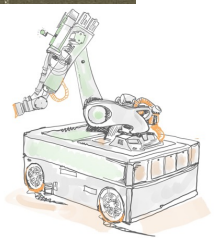
$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\delta} \end{bmatrix} = \begin{bmatrix} \cos\delta \\ \sin\delta \\ 0 \end{bmatrix} v + \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} w$$

So we can divide into 2 problems the pose estimate:

- Attitude
- Position







Differential drive kinematics

The differential-drive kinematics can be separated into:

$$\boxed{v(t) = \frac{v_R(t) + v_L(t)}{2}} \qquad \text{linear motion}$$

and

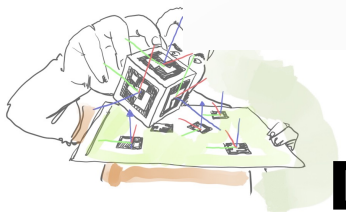
$$\boxed{\dot{\delta}(t) = \frac{v_R(t) - v_L(t)}{b}} \qquad \text{angular motion.}$$

Therefore,

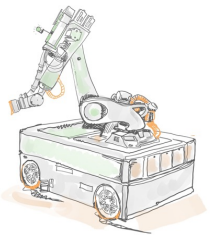
$$\boxed{\begin{bmatrix} v(t) \\ \dot{\delta}(t) \end{bmatrix} = \begin{bmatrix} \frac{1}{2} & \frac{1}{2} \\ \frac{1}{b} & -\frac{1}{b} \end{bmatrix} \begin{bmatrix} v_R(t) \\ v_L(t) \end{bmatrix}}$$

with the sign convention chosen here as

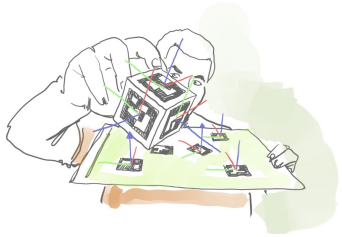
$$v_R > v_L \quad \downarrow \quad \dot{\delta} > 0.$$

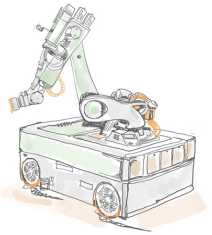


Differential drive kinematics



ATTITUDE ESTIMATION





1. Encoder increments

$$\Delta n_{R,k} = n_{R,k} - n_{R,k-1}$$

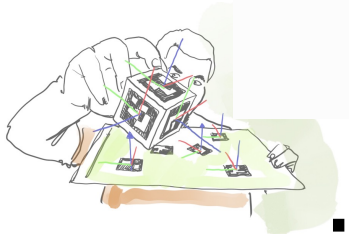
$$\Delta n_{L,k} = n_{L,k} - n_{L,k-1}$$

$$\Delta s_{R,k} = \frac{2\pi R_R}{N_R} \Delta n_{R,k}$$

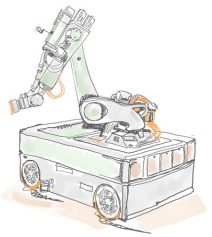
$$\Delta s_{L,k} = \frac{2\pi R_L}{N_L} \Delta n_{L,k}$$

$$\boxed{\Delta s_k^E = \frac{\Delta s_{R,k} + \Delta s_{L,k}}{2}}$$

$$\boxed{\Delta \delta_k^E = \frac{\Delta s_{R,k} - \Delta s_{L,k}}{b}}$$

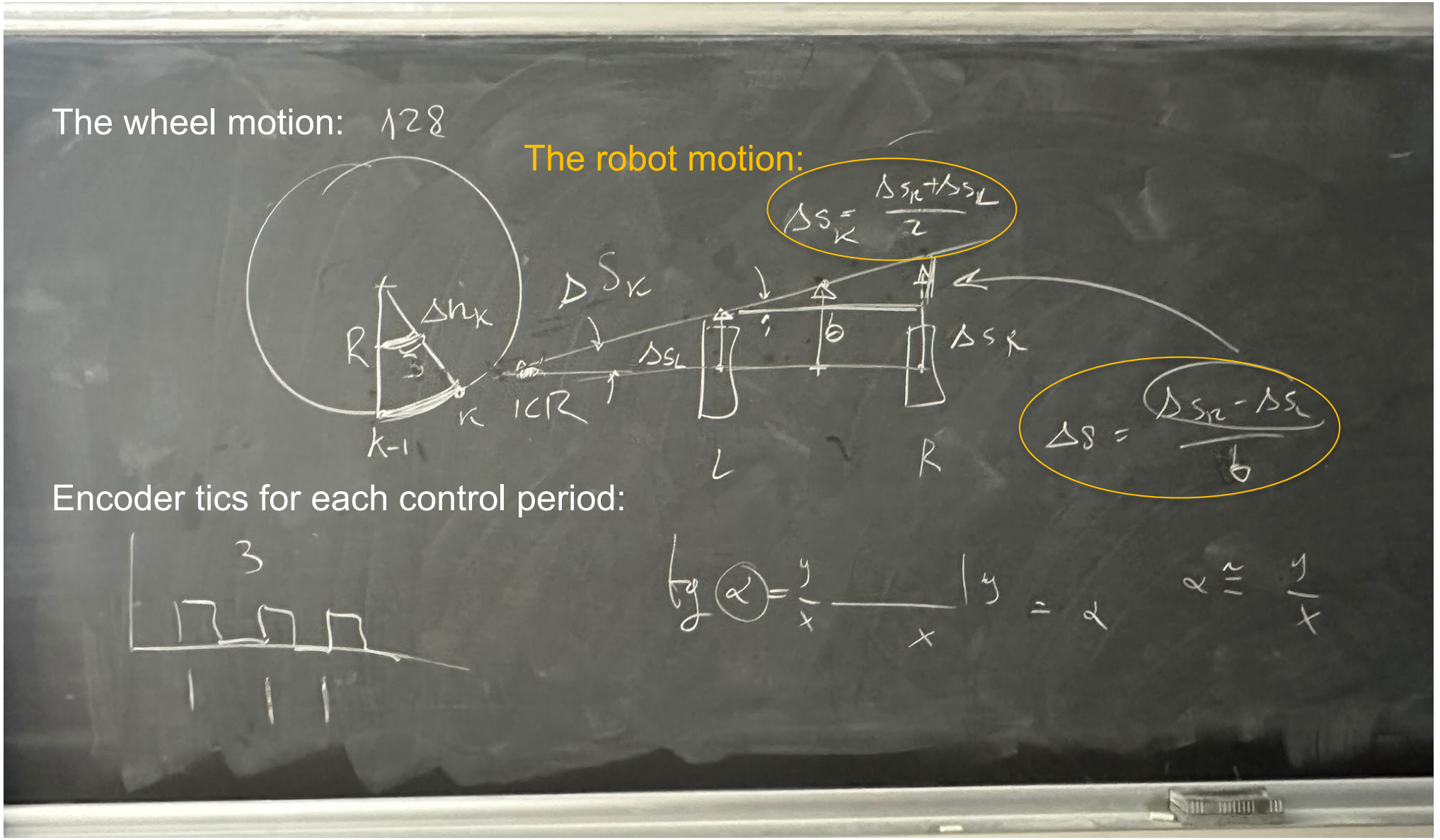


...

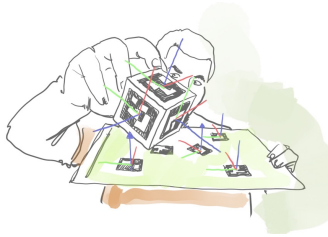


The wheel motion:

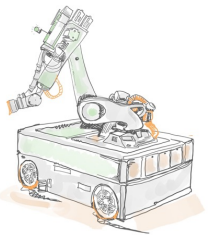
The robot motion:



Encoder tics for each control period:



From encoder increments to forward motion and attitude increments

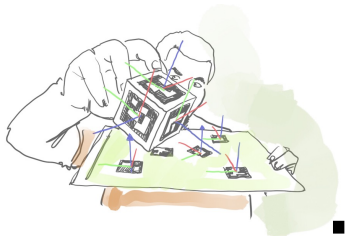


2. Gyroscope increment

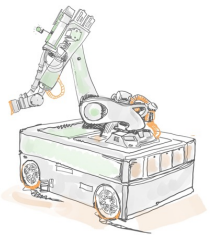
$$\omega_{G,k}$$

$$\boxed{\Delta\delta_k^G = T_c(\omega_{G,k} - b_G)}$$

You could estimate the bias b_G in real time with another MADS agent and pass the value to the filter agent



...



3. Continuous maneuver index

0: straight path
1: turning on place

$$\mu_k = \frac{|\Delta s_{R,k} - \Delta s_{L,k}|}{|\Delta s_{R,k}| + |\Delta s_{L,k}| + \varepsilon}$$

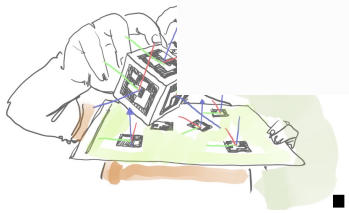
$$0 \leq \mu_k \leq 1$$

$$\rho_k = f_\rho \left(|\Delta s_{R,k}| + |\Delta s_{L,k}| \right)$$

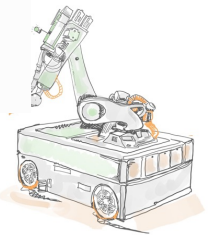
$$0 \leq \rho_k \leq 1$$

$$\xi_{R,k} = f_\xi(\mu_k, \rho_k)$$

$$\xi_{R,k} = \frac{\sigma^E_{\Delta\delta,k}}{\sigma^G_{\Delta\delta,k}}$$



...



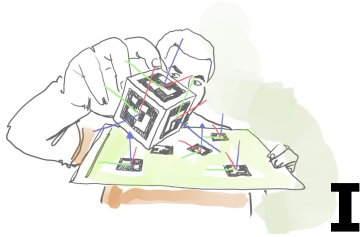
INSTITUTE OF PHYSICS PUBLISHING MEASUREMENT SCIENCE AND TECHNOLOGY

Meas. Sci. Technol. **14** (2003) 643–653 PII: S0957-0233(03)57271-X

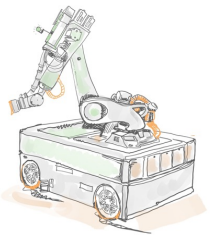
Sensor fusion of inertial-odometric navigation as a function of the actual manoeuvres of autonomous guided vehicles

Mariolino De Cecco

CISAS, Centre of Studies and Activities for Space, Via Venezia 1, 35131 Padova, Italy



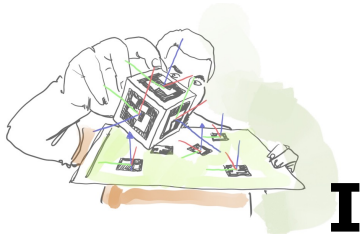
IoP 2003



$$DF(\Delta\delta_k^E, \Delta\delta_k^G) = \frac{(\sigma_{\Delta\delta}^E)^2 \Delta\delta_k^G + (\sigma_{\Delta\delta}^G)^2 \Delta\delta_k^E}{(\sigma_{\Delta\delta}^E)^2 + (\sigma_{\Delta\delta}^G)^2}. \qquad (5)$$

Actually, in order to fuse the data, only the ratio of the two uncertainties $\sigma_{\Delta\delta}^E / \sigma_{\Delta\delta}^G$, defined as ξ_R, is needed:

$$\Delta\delta_k = \frac{\Delta\delta_k^G}{\frac{1}{\xi_R^2} + 1} + \frac{\Delta\delta_k^E}{\xi_R^2 + 1}. \qquad (6)$$



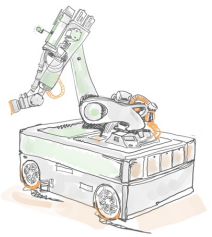
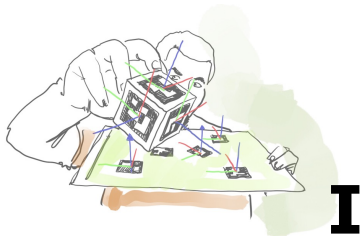


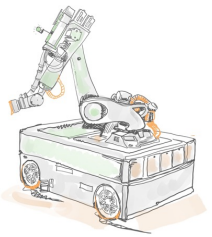
Table 1. The accuracy ratios for different vehicle manoeuvres.

Vehicle manoeuvre	Odometric accuracy: qualitative considerations	Inertial accuracy: qualitative considerations	Accuracy ratio ξ_R
Standing	High	Low	0
Curving with the steering angle blocked	Medium	High	2
Steering	Low	High	4
Straight path	High	Low	0.25

Table 2. Definition of the different vehicle manoeuvres as a function of encoder data.

Vehicle status	n_k	α_k	$(\alpha_k - \alpha_{k-1})/T_c$
Standing	$=0$	Any	Any
Curving with the steering angle blocked	$\neq 0$	$\geqslant 10°$	$<1°\ s^{-1}$
Steering	$\neq 0$	Any	$\geqslant 1°\ s^{-1}$
Straight path	$\neq 0$	$<10°$	$<1°\ s^{-1}$

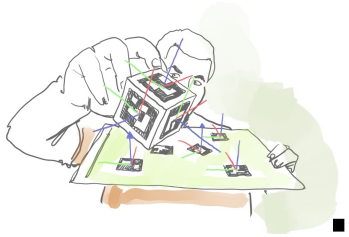




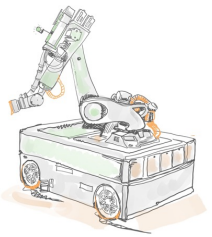
4. Encoder–gyro attitude fusion

$$\Delta\delta_k^{EG} = \frac{1}{1+\xi_{R,k}^2}\Delta\delta_k^{E} + \frac{\xi_{R,k}^2}{1+\xi_{R,k}^2}\Delta\delta_k^{G}$$

NOTE: in the paper of the 2003 I proposed a discrete values approach for ζ_R, you have to interpolate with a smooth function in order not to induce little steps/noise into the final estimate



...



$$\boxed{\mu_k = \frac{|\Delta s_{R,k} - \Delta s_{L,k}|}{|\Delta s_{R,k}| + |\Delta s_{L,k}| + \varepsilon}}$$

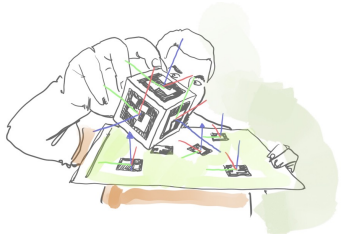
$$0 \leq \mu_k \leq 1$$

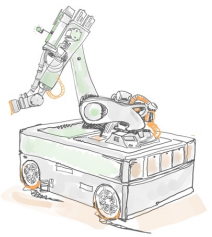
$$\boxed{\rho_k = f_\rho\left(|\Delta s_{R,k}| + |\Delta s_{L,k}|\right)}$$

$$0 \leq \rho_k \leq 1$$

$$\boxed{\xi_{R,k} = f_\xi(\mu_k, \rho_k)}$$

$$\xi_{R,k} = \frac{\sigma^E_{\Delta\delta,k}}{\sigma^G_{\Delta\delta,k}}$$

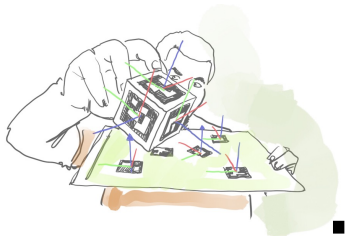




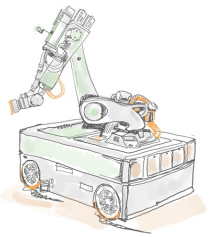
5. Relative attitude estimate

Encoder-gyro → $\boxed{\delta_k^{EG} = \delta_{k-1}^{EG} + \Delta\delta_k^{EG}}$

HTC → δ_k^{H}



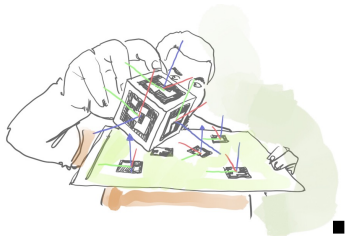
...



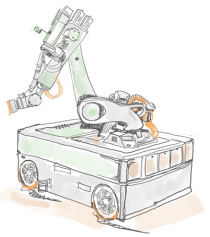
6. Attitude complementary filter

$$\boxed{\delta^F(s) = H_\delta^{HP}(s)\,\delta^{EG}(s) + H_\delta^{LP}(s)\,\delta^H(s)}$$

$$\boxed{H_\delta^{HP}(s) + H_\delta^{LP}(s) = 1}$$



...



6.1 Third-order continuous filters

$$A_\delta(s) = s^3 + a_{\delta,1}s^2 + a_{\delta,2}s + a_{\delta,3}$$

$$\boxed{B_\delta^{HP}(s) = s^3}$$

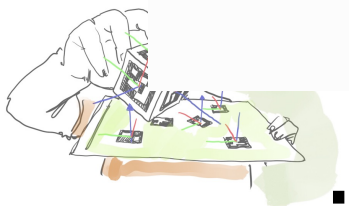
$$\boxed{H_\delta^{HP}(s) = \frac{s^3}{s^3 + a_{\delta,1}s^2 + a_{\delta,2}s + a_{\delta,3}}}$$

Complementarity condition

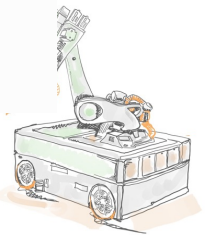
$$B_\delta^{LP}(s) = A_\delta(s) - B_\delta^{HP}(s)$$

$$\boxed{B_\delta^{LP}(s) = a_{\delta,1}s^2 + a_{\delta,2}s + a_{\delta,3}}$$

$$\boxed{H_\delta^{LP}(s) = \frac{a_{\delta,1}s^2 + a_{\delta,2}s + a_{\delta,3}}{s^3 + a_{\delta,1}s^2 + a_{\delta,2}s + a_{\delta,3}}}$$



...



```
fc_delta = 10;
Tc = 0.03;

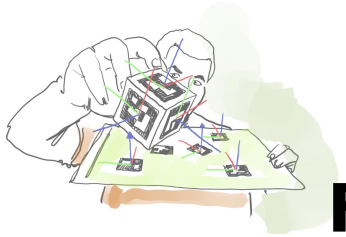
wc_delta = 2 * pi * fc_delta;

[Bh_delta, A_delta] = butter(3, wc_delta, 'high', 's');
Bl_delta = A_delta - Bh_delta;

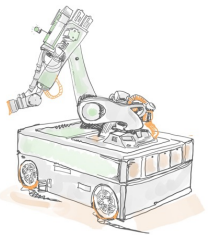
Hhp_delta_s = tf(Bh_delta, A_delta)
Hlp_delta_s = tf(Bl_delta, A_delta)

Hhp_delta_z = c2d(Hhp_delta_s, Tc, 'tustin')
Hlp_delta_z = c2d(Hlp_delta_s, Tc, 'tustin')

[bh_delta, ah_delta] = tfdata(Hhp_delta_z, 'v');
[bl_delta, al_delta] = tfdata(Hlp_delta_z, 'v');
```



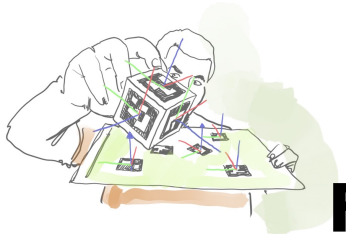
Filters discretization – matlab



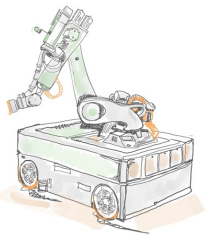
$$\delta_k^{HP} = -\sum_{i=1}^{3} a_{\delta,i}^{d} \delta_{k-i}^{HP} + \sum_{i=0}^{3} b_{\delta,i}^{HP} \delta_{k-i}^{EG}$$

$$\delta_k^{LP} = -\sum_{i=1}^{3} a_{\delta,i}^{d} \delta_{k-i}^{LP} + \sum_{i=0}^{3} b_{\delta,i}^{LP} \delta_{k-i}^{H}$$

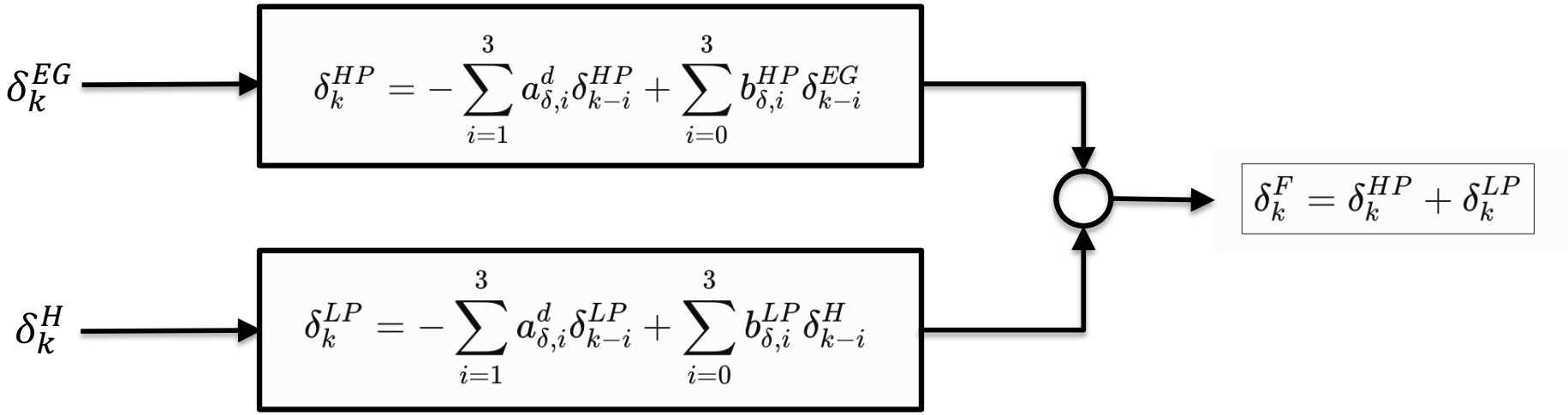
$$\boxed{\delta_k^{F} = \delta_k^{HP} + \delta_k^{LP}}$$

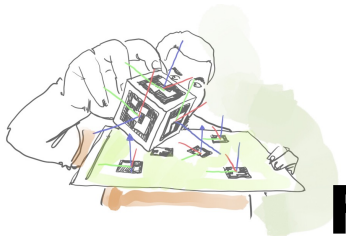


Filters discretization

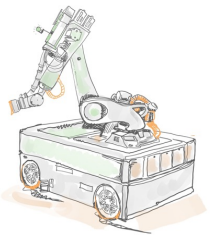


.. Same equations in complementary filter scheme:

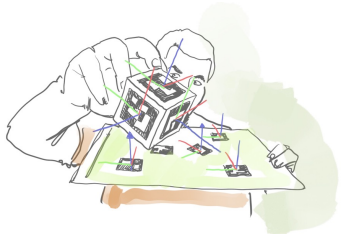


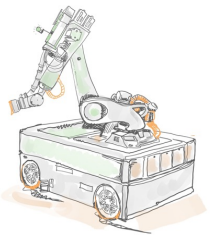


Filters discretization



POSITION ESTIMATION





7. Encoder position estimate

$$\boxed{\bar{\delta}_k^F = \frac{\delta_{k-1}^F + \delta_k^F}{2}}$$

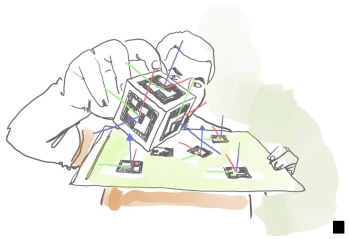
$$\boxed{x_k^E = x_{k-1}^E + \Delta s_k^E \cos \bar{\delta}_k^F}$$

$$\boxed{y_k^E = y_{k-1}^E + \Delta s_k^E \sin \bar{\delta}_k^F}$$

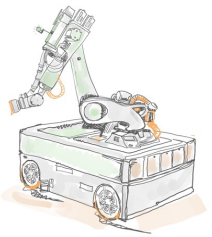
8. HTC position estimate

$$\boxed{x_k^H}$$

$$\boxed{y_k^H}$$



...

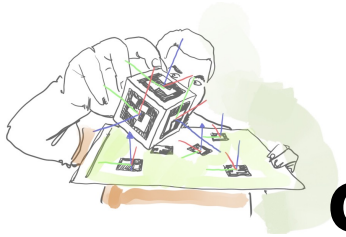


9. Position complementary filters

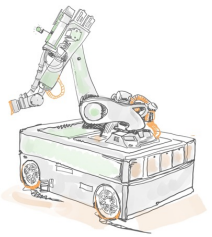
$$x^F(s) = H_p^{HP}(s)x^E(s) + H_p^{LP}(s)x^H(s)$$

$$y^F(s) = H_p^{HP}(s)y^E(s) + H_p^{LP}(s)y^H(s)$$

$$H_p^{HP}(s) + H_p^{LP}(s) = 1$$



Complementary filters



9.1 Third-order continuous filters

$$A_p(s) = s^3 + a_{p,1}s^2 + a_{p,2}s + a_{p,3}$$

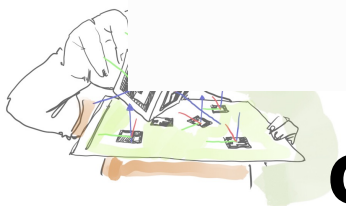
$$\boxed{B_p^{HP}(s) = s^3}$$

$$\boxed{H_p^{HP}(s) = \frac{s^3}{s^3 + a_{p,1}s^2 + a_{p,2}s + a_{p,3}}}$$

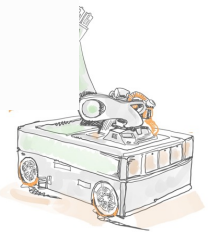
$$B_p^{LP}(s) = A_p(s) - B_p^{HP}(s)$$

$$\boxed{B_p^{LP}(s) = a_{p,1}s^2 + a_{p,2}s + a_{p,3}}$$

$$\boxed{H_p^{LP}(s) = \frac{a_{p,1}s^2 + a_{p,2}s + a_{p,3}}{s^3 + a_{p,1}s^2 + a_{p,2}s + a_{p,3}}}$$



Complementary filters



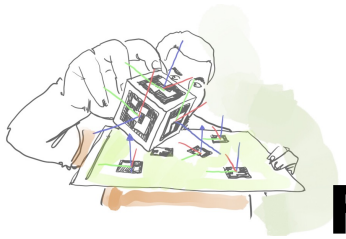
```
wc_p = 2*pi*fc_p;

[Bh_p, A_p] = butter(3,wc_p, 'high', 's');
Bl_p = A_p - Bh_p;

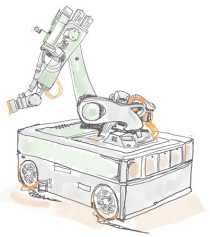
Hhp_p_s = tf(Bh_p, A_p);
Hlp_p_s = tf(Bl_p, A_p);

Hhp_p_z = c2d(Hhp_p_s, Tc, 'tustin');
Hlp_p_z = c2d(Hlp_p_s, Tc, 'tustin');

[bh_p,ah_p] = tfdata(Hhp_p_z, 'v');
[bl_p,al_p] = tfdata(Hlp_p_z, 'v');
```



Filters discretization - matlab



10. x recurrence

$$x_k^{HP} = -\sum_{i=1}^{3} a_{p,i}^{d} x_{k-i}^{HP} + \sum_{i=0}^{3} b_{p,i}^{HP} x_{k-i}^{E}$$

$$x_k^{LP} = -\sum_{i=1}^{3} a_{p,i}^{d} x_{k-i}^{LP} + \sum_{i=0}^{3} b_{p,i}^{LP} x_{k-i}^{H}$$

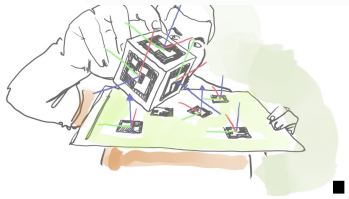
$$\boxed{x_k^{F} = x_k^{HP} + x_k^{LP}}$$

11. y recurrence

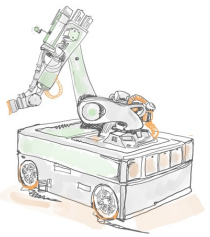
$$y_k^{HP} = -\sum_{i=1}^{3} a_{p,i}^{d} y_{k-i}^{HP} + \sum_{i=0}^{3} b_{p,i}^{HP} y_{k-i}^{E}$$

$$y_k^{LP} = -\sum_{i=1}^{3} a_{p,i}^{d} y_{k-i}^{LP} + \sum_{i=0}^{3} b_{p,i}^{LP} y_{k-i}^{H}$$

$$\boxed{y_k^{F} = y_k^{HP} + y_k^{LP}}$$



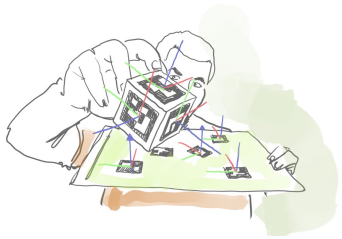
...

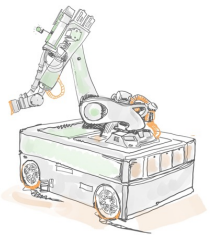


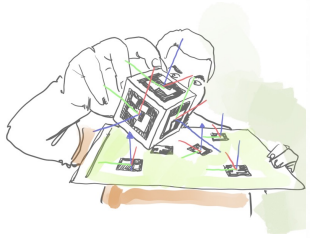
The whole algorithm flowchart

CURRENT FUSED POSE

$$\mathbf{x}_k^F = \begin{bmatrix} x_k^F \\ y_k^F \\ \delta_k^F \end{bmatrix}$$







Wheel Encoders
$n_{L,k},\ n_{R,k}$

Gyroscope
$\omega_{G,k}$

HTC
$\delta_k^H,\ x_k^H,\ y_k^H$

Perception loop
$k-1 \rightarrow k$
(T_c)

1. Encoder increments

$\Delta n_{L,k},\ \Delta n_{R,k}$

$\Delta s_{L,k},\ \Delta s_{R,k}$

$\Delta s_k^E,\ \Delta \delta_k^E$

2. Gyro increment

$\Delta \delta_k^G = T_c\,(\omega_{G,k} - b_G)$

3. Maneuver-dependent weights

$\mu_k,\ \rho_k$

$\xi_{R,k} = f(\mu_k, \rho_k)$

4. Encoder–gyro attitude fusion

$\Delta \delta_k^{EG} = w_E\,\Delta \delta_k^E + w_G\,\Delta \delta_k^G$

5. Relative attitude estimate

$\delta_k^{EG} = \delta_{k-1}^{EG} + \Delta \delta_k^{EG}$

6. Attitude complementary filter

$\delta_k^F = H_p^{HP}(z)\{\delta_k^{EG}\} + H_p^{LP}(z)\{\delta_k^H\}$

7. Encoder position estimate

$x_k^E = x_{k-1}^E + \Delta s_k^E \cos\left(\bar{\delta}_k^F\right)$

$y_k^E = y_{k-1}^E + \Delta s_k^E \sin\left(\bar{\delta}_k^F\right)$

8. Position complementary filter (x)

$x_k^F = H_p^{HP}(z)\{x_k^E\} + H_p^{LP}(z)\{x_k^H\}$

9. Position complementary filter (y)

$y_k^F = H_p^{HP}(z)\{y_k^E\} + H_p^{LP}(z)\{y_k^H\}$

10. Fused pose at k

$x_k^F,\ y_k^F,\ \delta_k^F$

11. Update stored states for next iteration

$x_{k-1} \leftarrow x_k,\quad y_{k-1} \leftarrow y_k,\quad \delta_{k-1} \leftarrow \delta_k$

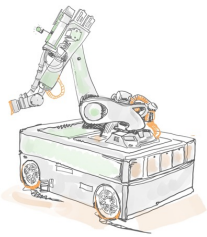
(and filter histories)

Previous state $(k-1)$

$x_{k-1},\ y_{k-1},\ \delta_{k-1}$

(filter histories)

Encoder processing
Gyro processing
Maneuver weighting
Attitude estimation
Position estimation
State update



Notation tables — robot, sensors and increments

Editable PowerPoint tables for the complementary-filtering exercise

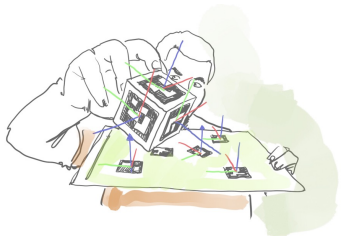
E = encoder / odometry | G = gyroscope | H = HTC absolute | F = fused

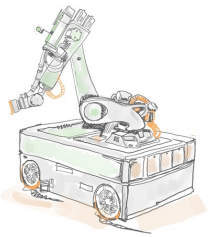
General notation and pose

Symbol	Meaning
k	Current discrete-time sample / Perception loop
k−1	Previous discrete-time sample
t_k	Time corresponding to sample k
T_c = t_k − t_(k−1)	Perception-loop sampling period
x_k, y_k	Robot planar position at sample k
δ_k	Robot attitude / yaw angle at sample k
X_k = [x_k, y_k, δ_k]^T	Robot pose vector
P_r	Robot reference point
b	Distance between right and left driving wheels
Δ(·)_k	Increment of a quantity over t_(k−1) → t_k

Wheel encoders and differential-drive kinematics

Symbol	Meaning
R_R, R_L	Right- and left-wheel radii
N_R, N_L	Right and left encoder counts per revolution
n_(R,k), n_(L,k)	Right and left cumulative encoder counts
Δn_(R,k), Δn_(L,k)	Right and left encoder count increments
ω_R(t), ω_L(t)	Right and left wheel angular velocities
v_R(t), v_L(t)	Right and left wheel linear velocities
v(t)	Robot longitudinal velocity
δ(t)	Robot yaw rate
Δs_(R,k), Δs_(L,k)	Signed right and left wheel displacements
Δs_k^E	Encoder estimate of longitudinal displacement
Δδ_k^E	Encoder estimate of attitude increment





Notation tables — gyro, maneuver weighting and attitude

Variables used before and after the encoder–gyro attitude fusion

Superscripts and sensor labels

Symbol	Meaning
E	Encoder / odometric estimate
G	Gyroscope estimate
H	HTC absolute measurement
EG	Encoder–gyroscope fused estimate
F	Final fused estimate
HP	High-pass filter / branch
LP	Low-pass filter / branch

Gyroscope

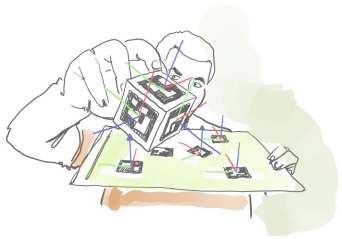
Symbol	Meaning
$\omega_{(G,k)}$	Gyroscope measured yaw rate
b_G	Gyroscope zero-rate bias
$\Delta\delta_k^G$	Gyroscope estimate of attitude increment
$\Delta\delta_k^G = T_c(\omega_{(G,k)} - b_G)$	Bias-corrected discrete increment

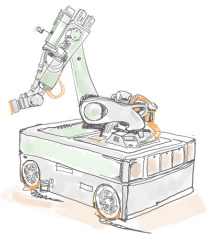
Continuous maneuver-dependent fusion

Symbol	Meaning
μ_k	Continuous turning / differential-wheel-motion index
ρ_k	Continuous vehicle-motion index
ε	Small positive constant to avoid division by zero
$\sigma_{(\Delta\delta,k)}^E$	Standard uncertainty of encoder attitude increment
$\sigma_{(\Delta\delta,k)}^G$	Standard uncertainty of gyro attitude increment
$\xi_{(R,k)}$	Encoder-to-gyro attitude uncertainty ratio
$f_\rho(\cdot)$	Continuous function used to compute ρ_k
$f_\xi(\cdot)$	Continuous function used to compute $\xi_{(R,k)}$
$w_{(E,k)}$	Encoder weight in attitude-increment fusion
$w_{(G,k)}$	Gyroscope weight in attitude-increment fusion
$\Delta\delta_k^{EG}$	Encoder–gyro fused attitude increment

Attitude estimates

Symbol	Meaning
δ_k^{EG}	Relative attitude obtained by integrating $\Delta\delta_k^{EG}$
δ_k^H	Absolute attitude measured by HTC
δ_k^F	Complementary-filter fused attitude
$\Delta\delta_k^F$	Increment of fused attitude
$\bar{\delta}_k^F$	Representative / mean fused attitude over $k-1 \to k$
$\mathrm{wrap}(\cdot)$	Angle wrapping operation, e.g. to $(-\pi, \pi]$





Notation tables — position estimates and continuous filters

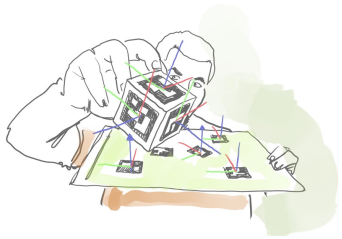
Same position complementary filter is applied independently to x and y

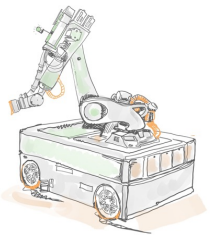
Position estimates

Symbol	Meaning
x_k^E, y_k^E	Encoder dead-reckoning position estimate
x_k^H, y_k^H	Absolute HTC position measurement
x_k^F, y_k^F	Complementary-filter fused position
p_k^E = [x_k^E, y_k^E]^T	Encoder position vector
p_k^H = [x_k^H, y_k^H]^T	HTC position vector
p_k^F = [x_k^F, y_k^F]^T	Fused position vector
x_k^E = x_(k−1)^E + Δs_k^E cos(δ¯k^F)	Encoder x propagation
y_k^E = y_(k−1)^E + Δs_k^E sin(δ¯k^F)	Encoder y propagation

Continuous complementary-filter notation

Symbol	Meaning
s	Laplace-transform variable
H_δ^HP(s), H_δ^LP(s)	Attitude high-pass and low-pass filters
H_p^HP(s), H_p^LP(s)	Position high-pass and low-pass filters
A_δ(s), A_p(s)	Common denominator polynomials
B_δ^HP(s), B_δ^LP(s)	Attitude HP and LP numerator polynomials
B_p^HP(s), B_p^LP(s)	Position HP and LP numerator polynomials
a_(δ,1..3)	Continuous attitude-filter denominator coefficients
a_(p,1..3)	Continuous position-filter denominator coefficients
f_(c,δ), f_(c,p)	Attitude and position cutoff frequencies [Hz]
ω_(c,δ), ω_(c,p)	Cutoff angular frequencies: 2πf_c
B^HP(s)=s^3	Third-order HP numerator used in this exercise
B^LP(s)=A(s)−B^HP(s)	Complementary numerator construction





Notation tables — discrete filters and internal states

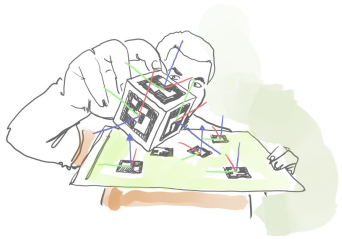
C++ / MADS recurrence variables after MATLAB c2d conversion

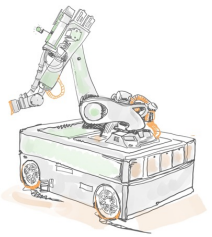
Discrete complementary-filter notation

Symbol	Meaning
z	Z-transform variable
z^{-1}	One-sample delay
$H^{HP}(z)$	Discrete high-pass transfer function
$H^{LP}(z)$	Discrete low-pass transfer function
a_i^d	Discrete denominator coefficient
b_i^{HP}	Discrete HP numerator coefficient
b_i^{LP}	Discrete LP numerator coefficient
$i = 0,\dots,3$	Numerator coefficient index
$i = 1,\dots,3$	Denominator coefficient index
$c2d(\cdot)$	MATLAB conversion from continuous to discrete TF

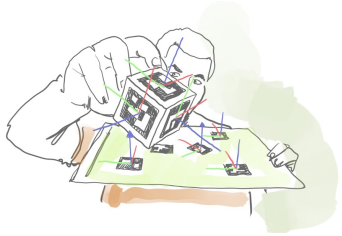
Internal filter outputs / histories

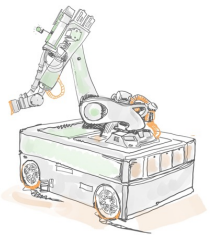
Symbol	Meaning
δ_k^{HP}	HP-filtered encoder–gyro attitude
δ_k^{LP}	LP-filtered HTC attitude
$\delta_k^F = \delta_k^{HP} + \delta_k^{LP}$	Fused attitude output
x_k^{HP}	HP-filtered encoder x-position estimate
x_k^{LP}	LP-filtered HTC x-position estimate
$x_k^F = x_k^{HP} + x_k^{LP}$	Fused x-position output
y_k^{HP}	HP-filtered encoder y-position estimate
y_k^{LP}	LP-filtered HTC y-position estimate
$y_k^F = y_k^{HP} + y_k^{LP}$	Fused y-position output
history buffers	Last 3 inputs and outputs for each branch





MADS AGENT AND DATA AT DISPOSAL





It must be a filter plugin with the following **input** and **output**:

INPUT TOPICS

- htc
- calibrated_imu
- driver

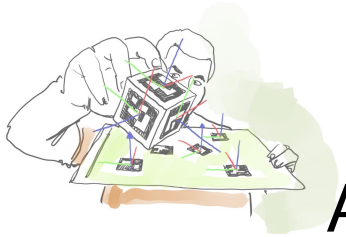
OUTPUT TOPIC

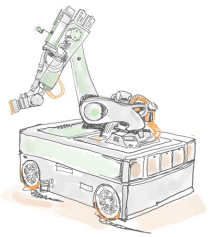
- ego_state_(number of your group)

PARAMETERS (in set_params method)

- gyroscope uncertainty: **_params["imu"][“sigma_gyro”];** (*unit of measure*: radians per second)
- wheel’s radius**: _params[“wheel_radius”];** *(unit of measure: meter)*
- baseline: **_params[“wheel_base”];** *(unit of measure: meter)*

check for the existence of these fields is suggested





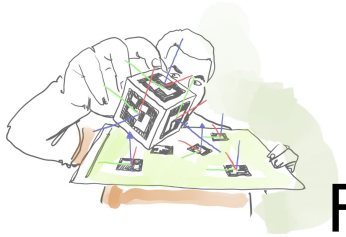
Agents to develop

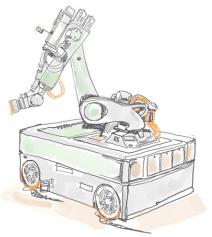
Cumulative count of the incremental encoders

- *Unit of measure*: tics
- *Counts per revolution*: 4096

driver 0.019 s (0.008 s±0.012 s)


```
{
  "agent_id": "",
  "currents": {
    "left": 0,
    "right": 0
  },
  "encoders": {
    "left": 0,
    "right": 0
  },
  "hostname": "raspberrypi",
  "issue": "no issue",
  "speeds": {
    "left": 0,
    "right": 0
  },
  "temperature MCU": 28,
  "temperature motor 1": 20,
  "temperature motor 2": 20,
  "timecode": 23700.8,
  "timestamp": {
    "$date": "2026-10-07T06:35:00.817+0200"
  },
  "voltage": 49.2
}
```



Format of the input topics: **driver**

This structure is the same for the following fields: «**Htracker**» for the midpoint between the wheels and "**Htracker_raw**" for the actual tracker position

4x4 homogeneous transformation coordinates

3x3 rotation transformation coordinates

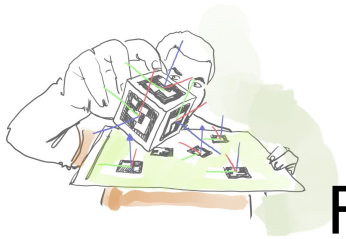
Euler angles

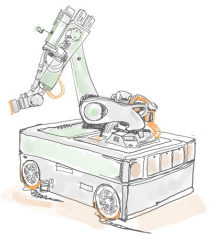
- Unit of measure: radians

Position coordinates

- Unit of measure: meters

```
htc 0.053 s (0.100 s±0.014 s)
{
  "Htracker": {
    "pose": {
      "H": [
        [
          0.47680988907814026,
          0.8767042756080627,
          -0.06357640773057938,
          -1.020214557647705
        ],
        [
          -0.8738012313842773,
          0.4648854732513428,
          -0.1426636427640915,
          0.13321265578269958
        ],
        [
          -0.0955180674791336,
          0.1235765889286995,
          0.9877272248268127,
          -1.4949774742126465
        ],
        [
          0.0,
          0.0,
          0.0,
          1.0
        ]
      ],
      "R": [
        [
          0.4782273769378662,
          0.8750471472740173,
          -0.07477361708879471
        ],
        [
          -0.8733378648757935,
          0.4648539125919342,
          -0.14557364583015442
        ],
        [
          -0.0926249772310257,
          0.1349199265241623,
          0.9865177869796753
        ]
      ],
      "attitude": [
        0.14650584757328033,
        -0.07484346628189087,
        -1.070631742477417
      ],
      "position": [
        -1.020684003829956,
        0.1325424313545227,
        -1.5009939670562744
      ]
    }
  },
```





Format of the input topics: **htc**

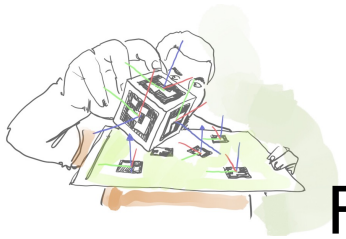
Acceleration on the 3 axis (shown in the reference system slide)

- *Unit of measure*: $^{m}/_{s^2}$

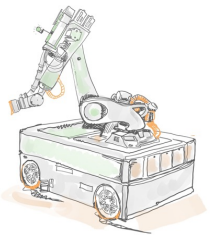
Angular velocity on the ‘z’ axis

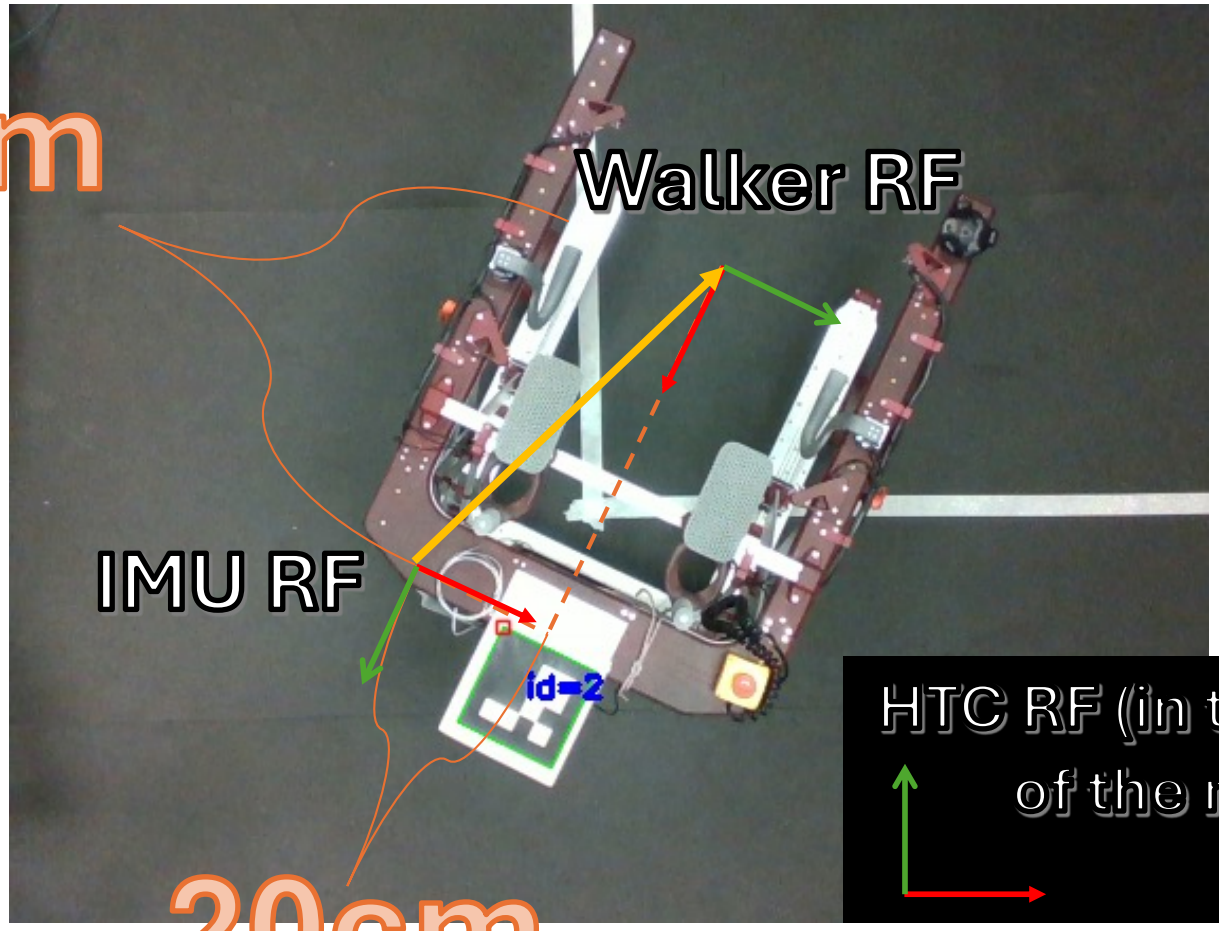
- *Unit of measure*: radians per second (rad/s)

```
calibrated_imu 0.015 s (0.035 s±0.018 s)
{
  "acc_x": 0.05833699507419951,
  "acc_y": -6.600434186995961,
  "agent_id": "",
  "gyro": 0.0013135209286976235,
  "hostname": "raspberrypi",
  "timecode": 28190.5,
  "timestamp": {
    "$date": "2026-10-07T07:49:50.512+0200"
  }
}
```

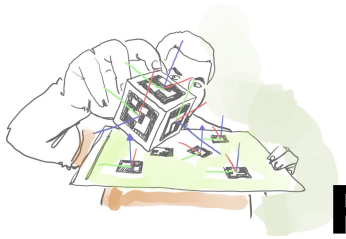


Format of the input topics: **calibrated_imu**

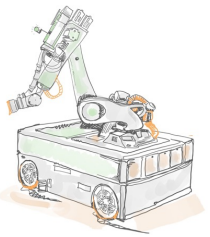




Angular velocity is given accordingly to walker RF



Reference Frame

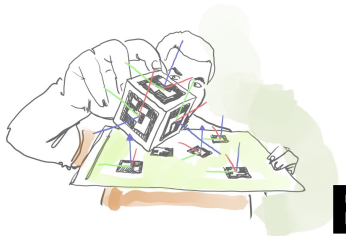


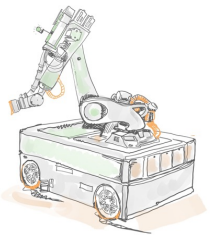
Expected Output

It will also be provided a similar topic with data computed from another algorithm that you can use **JUST** for validation of your complementary filter

4x4 homogeneous transformation coordinates

```
ego_state 0.044 s (0.013 s±0.021 s)
{
  "H": [
    [
      0.8618545925267233,
      0.5071554607224454,
      0.0,
      -0.8101077319396839
    ],
    [
      -0.5071554607224454,
      0.8618545925267233,
      0.0,
      0.9796040128743152
    ],
    [
      0.0,
      0.0,
      1.0,
      0.0
    ],
    [
      0.0,
      0.0,
      0.0,
      1.0
    ]
  ],
```





Add a section to your ini file and run the command:

```
mads play --input FILE.bag --rate YOUR_RATE
```

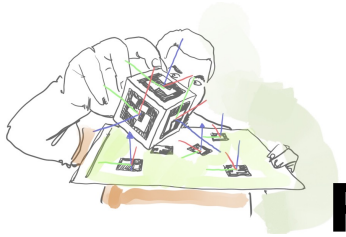
plays the recorded data in FILE.bag at the specified rate. (A rate of 1 is normal speed; 0.5 is half speed, and 2 is double speed)

Run a broker to send settings to play agent (this keeps to be valid if you want to run any other agent in the next slides):

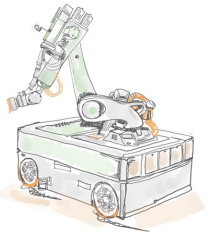
```
mads broker
```

Run the following command to export the file as a list of json:

```
mads bag export -f FILE.bag -o NAME.jsonl
```



Run collected data at home

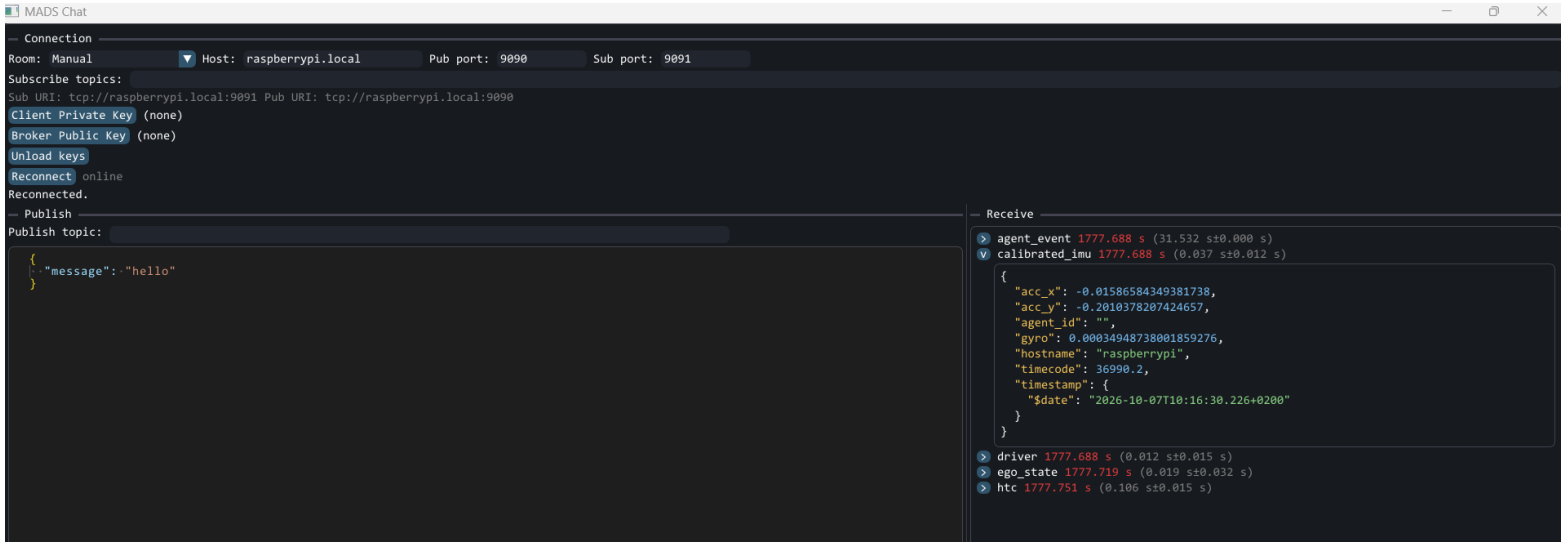


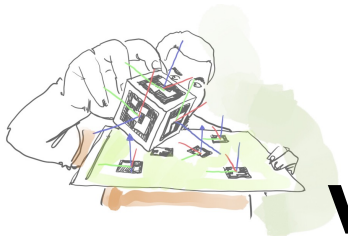
Install mads-chat

```
mads package –i mads-chat
```

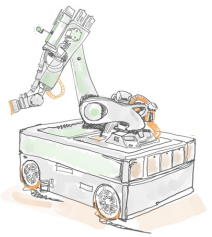
launch the following command to visualize the data and topics published

```
mads-chat
```



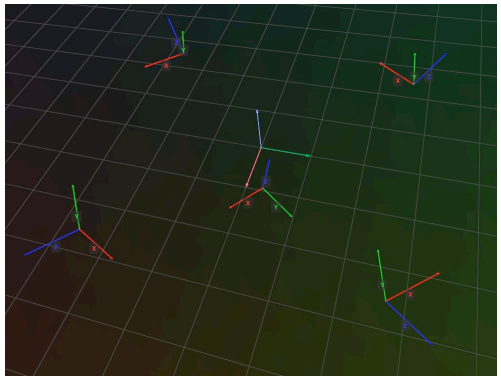


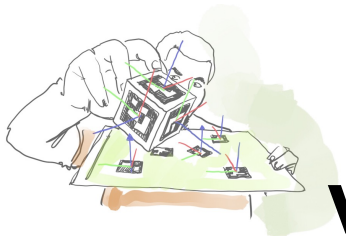
Visualization: mads-chat



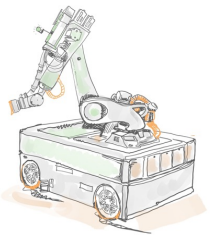
- Follow the instructions in the rerunner_plugin repository.
- Add a section to your .ini file with a name of your choice (for example, NAME_1). Include the following fields:
  - *sub_topic*: the topic containing the data you want to visualize.
  - *keypath*: the path to the data within the message that you want to plot.
  - *hmat_keypaths*: the paths to the elements of a 4 × 4 matrix used to visualize an object's pose.
- Run a rerunner with the name you chose:

```
mads sink rerunner.plugin –n NAME_1
```

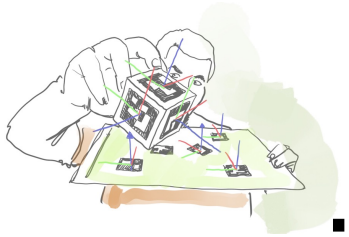




Visualization: rerunner



- Inserire indicazioni sull'agente da sviluppare:
- Input
- Output
- Dati a disposizione
- Info su replay da usare a casa x testare sw sviluppato
- …



...

